



INT305 Machine Learning
Lecture 7
Decision Trees & Bias-Variance Decomposition

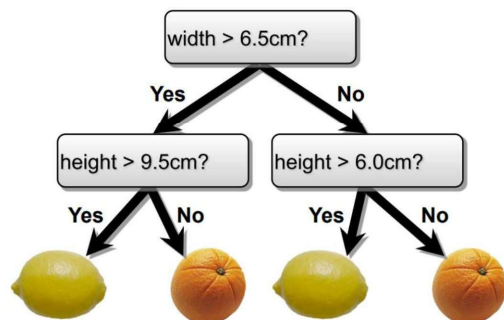
Siyue Yu
Department of Intelligence Science
siyue.yu02@xjtlu.edu.cn

Today

- **Decision Trees**
 - Simple but powerful learning algorithm 简单但强大的学习算法
 - Used widely in Kaggle competitions 广泛用于Kaggle竞赛
 - Let us motivate concepts from information theory (entropy, mutual information, etc.) 让我们从信息论（熵，互信息等）中激发概念。
- **Bias-variance decomposition**
 - Let us motivate methods for combining different classifiers.

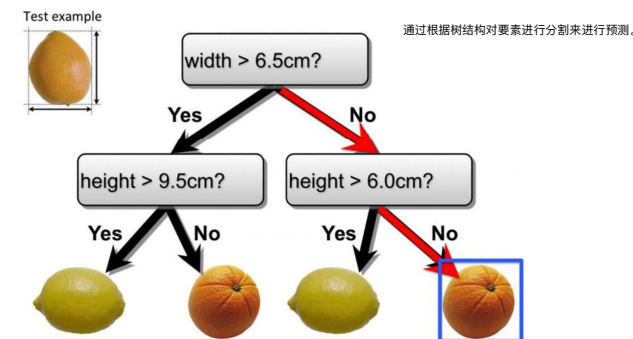
DecisionTrees

- Make predictions by splitting on features according to a tree structure.



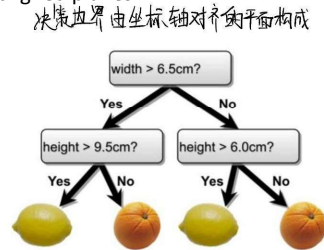
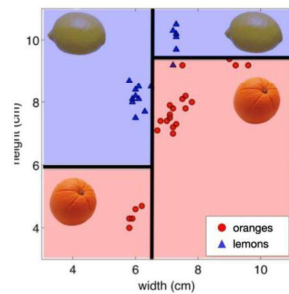
DecisionTrees

- Make predictions by splitting on features according to a tree structure.

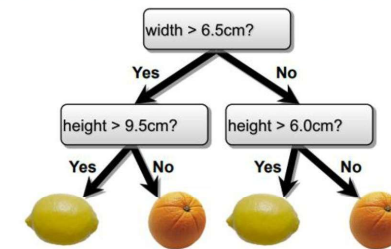


Decision Trees— Continuous Features

- Split *continuous features* by checking whether that feature is greater than or less than some threshold.
通过检查特征是否大于或小于某个阈值来拆分连续特征。
- Decision boundary is made up of axis-aligned planes.
决策边界由坐标轴对齐的平面构成



Decision Trees



- Internal nodes test a **feature**
- Branching is determined by the **feature value**
- Leaf nodes are **outputs** (predictions)

内部节点测试特征
分支由特征值决定
叶节点是输出（预测）

Decision Trees—Classification and Regression

决策树可以做两种工作
分类与回归

从“树根”走到“树叶”的每一条路径，都定义了输入空间中的一个区域 R_m

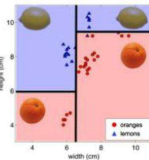
- Each path from root to a leaf defines a region R_m of input space
- Let $\{(x^{(m_1)}, t^{(m_1)}), \dots, (x^{(m_k)}, t^{(m_k)})\}$ be the training examples that fall into R_m

- Classification tree** (we will focus on this):

- discrete output 预测一个离散的值
- leaf value y^m typically set to the most common value in $\{t^{(m_1)}, \dots, t^{(m_k)}\}$ 树叶所代表的变量常被设置成当前区域内最常见的值

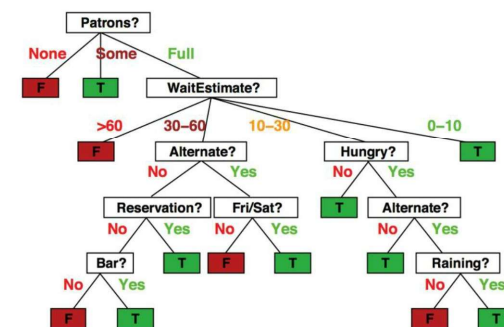
- Regression tree:**

- Continuous output 预测一个连续的值
- leaf value y^m typically set to the mean value in $\{t^{(m_1)}, \dots, t^{(m_k)}\}$ 树叶的值通常设置成该区域中所有样本的平均值



Decision Trees—Discrete Features

- Will I wait at this restaurant for dinner?



DecisionTrees—DiscreteFeatures

- Split discrete features into a partition of possible values. 将离散特征分割为可能值的划分。

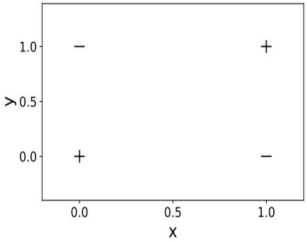
Example	Input Attributes										Goal WillWait
	Alt	Bar	Fri	Hun	Pat	Price	Rain	Res	Type	Est	
x ₁	Yes	No	No	Yes	Some	\$\$\$	No	Yes	French	0-10	y ₁ = Yes
x ₂	Yes	No	No	Yes	Full	\$	No	No	Thai	30-60	y ₂ = No
x ₃	No	Yes	No	No	Some	\$	No	No	Burger	0-10	y ₃ = Yes
x ₄	Yes	No	Yes	Yes	Full	\$	Yes	No	Thai	10-30	y ₄ = Yes
x ₅	Yes	No	Yes	No	Full	\$\$\$	No	Yes	French	>60	y ₅ = No
x ₆	No	Yes	No	Yes	Some	\$	Yes	Yes	Italian	0-10	y ₆ = Yes
x ₇	No	Yes	No	No	None	\$	Yes	No	Burger	0-10	y ₇ = No
x ₈	No	No	No	Yes	Some	\$	Yes	Yes	Thai	0-10	y ₈ = Yes
x ₉	No	Yes	Yes	No	Full	\$	Yes	No	Burger	>60	y ₉ = No
x ₁₀	Yes	Yes	Yes	Yes	Full	\$\$\$	No	Yes	Italian	10-30	y ₁₀ = No
x ₁₁	No	No	No	No	None	\$	No	No	Thai	0-10	y ₁₁ = No
x ₁₂	Yes	Yes	Yes	Yes	Full	\$	No	No	Burger	30-60	y ₁₂ = Yes

Features:

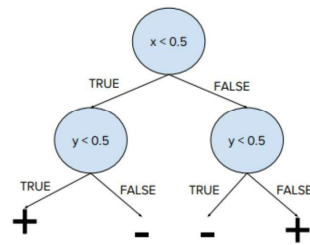
1. Alternate: whether there is a suitable alternative restaurant nearby.
2. Bar: whether the restaurant has a comfortable bar area to wait in.
3. Fri/Sat: true on Fridays and Saturdays.
4. Hungry: whether we are hungry.
5. Patrons: how many people are in the restaurant (values are None, Some, and Full).
6. Price: the restaurant's price range (\$, \$\$, \$\$\$).
7. Raining: whether it is raining outside.
8. Reservation: whether we made a reservation.
9. Type: the kind of restaurant (French, Italian, Thai or Burger).
10. WaitEstimate: the wait estimated by the host (0-10 minutes, 10-30, 30-60, >60).

Attempts

The drawing below shows a dataset. Each example in the dataset has two inputs features x and y , and maybe classified as a positive example (labelled +) or a negative example (labelled -). Draw a decision tree which correctly classifies each example in the dataset.



Attempts



Learning Decision Trees

对于任何训练集，我们都能构建一个为每个训练点精确生成单一叶节点的决策树，但其泛化能力很可能不佳。

过拟合

- For any training set we can construct a decision tree that has exactly the one leaf for every training point, but it probably won't generalize. 这种所谓的准确分类能力毫无用处。
 - Decision trees are universal function approximators. 决策树是通用函数逼近器。
- But, finding the smallest decision tree that correctly classifies a training set is NP complete. 寻找能够正确分类训练集的最小决策树是一个NP完全问题。
 - If you are interested, check: Hyafil & Rivest'76.
- So, how do we construct a useful decision tree?

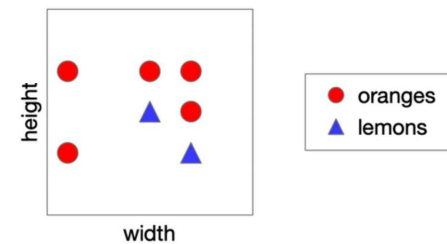
Learning Decision Trees

- Resort to a **greedy heuristic**:
 - Start with the whole training set and an empty decision tree.
 - Pick a feature and candidate split that would most reduce the loss.
 - Split on that feature and recurse on subpartitions.
- Which loss should we use?
 - Let's see if misclassification rate is a good loss.

从整个训练集和一个空的决策树开始。
 选择一个能最大程度减少损失的特征和候选划分点。
 在该特征上进行区分，并在子分区上进行递归执行。

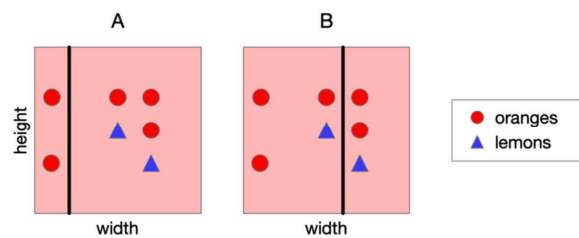
Choosing a Good Split

- Consider the following data. Let's split on width.



Choosing a Good Split

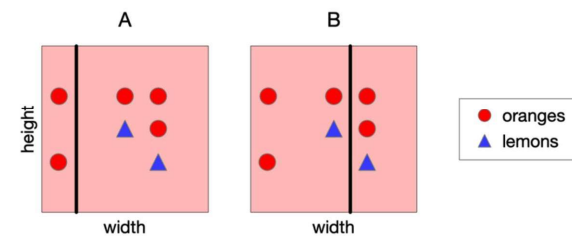
- Recall: classify by majority.



- A and B have the same misclassification rate, so which is the best split? Vote!

Choosing a Good Split

- A feels like a better split, because the left-hand region is very certain about whether the fruit is an orange. A的结果更好, 因为A至少能确定左侧的一定是 oranges



- Can we quantify this?

Choosing a Good Split

测试给定的叶子区域 的预测不确定性

- How can we quantify uncertainty in prediction for a given leaf node?
 - If all examples in leaf have same class: good, low uncertainty 如果在分类的一侧有完全相同的类
 - If each class has same amount of examples in leaf: bad, high uncertainty
- Idea:** use counts at leaves to define probability distributions; use a probabilistic notion of uncertainty to decide splits.
用叶子节点里的“计数”来定义“概率分布”；然后用一个基于概率的“不确定性”概念来决定如何分裂。
例如：好情况 $P(1) = \frac{10}{10} = 1.0$ $P(0) = \frac{0}{10} = 0.0$
- A brief detour through information theory...
然后需要定义一个数学公式 公式的输入就是上面的概率，得到一个不确定性得分。像上面的情况 得分 为 0

• 熵 (Entropy) 就是那个能衡量“混乱程度” (即不确定性) 的数学公式。
• 决策树在构建时，会评估所有可能的分裂方式 (比如“按年龄 > 30岁”分，还是“按收入 > 5万”分)。
• 它会选择那个能让分裂后的子节点“熵”的总和最小 (也就是“纯度”最高) 的分裂方式。这个“熵减少的量”通常被称为信息增益 (Information Gain)。
确定性越高

Quantifying Uncertainty

离散随机变量的熵是一个数值，用于量化其可能结果中固有的不确定性

- The **entropy** of a discrete random variable is a number that quantifies the **uncertainty** inherent in its possible outcomes.
- The mathematical definition of entropy that we give in a few slides may seem arbitrary, but it can be motivated axiomatically.
 - If you're interested, check: *Information Theory* by Robert Ash.
- To explain entropy, consider flipping two different coins...

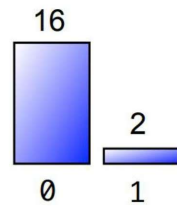
We Flip Two Different Coins 抛硬币

Sequence 1:

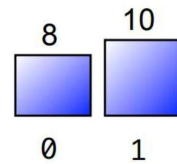
0 0 0 1 0 0 0 0 0 0 0 0 0 0 1 0 0 ... ?

Sequence 2:

0 1 0 1 0 1 1 1 0 1 0 0 1 1 0 1 0 1 ... ?



versus



Quantifying Uncertainty 计算不确定性

- The entropy of a loaded coin with probability p of heads is given by

$$-p \log_2(p) - (1-p) \log_2(1-p)$$



$$-\frac{8}{9} \log_2 \frac{8}{9} - \frac{1}{9} \log_2 \frac{1}{9} \approx \frac{1}{2}$$

$$-\frac{4}{9} \log_2 \frac{4}{9} - \frac{5}{9} \log_2 \frac{5}{9} \approx 0.99$$

- Notice: the coin whose outcomes are more certain has a lower entropy
熵越低, 确定性越高.
- In the extreme case $p = 0$ or $p = 1$, we were certain of the outcome before observing.

So, we gained no certainty by observing it, i.e., entropy is 0.

指对概率为0或1的情况, entropy一定是0, 因为非常确定

Quantifying Uncertainty

熵 (Entropy) = 预期的信息量 (Expected Information Content)

- “预期”在概率论中就是指“平均值”。
- 所以，熵就是“平均的意外程度”。

- Can also think of **entropy** as the expected information content of a random draw from a probability distribution.

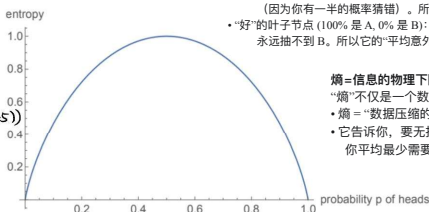
假设对于一个公平的硬币，
正反面是 50% 概率。

$$\begin{aligned} H &= -(P(A) \cdot \log_2(P(A)) + P(B) \cdot \log_2(P(B))) \\ &= -(0.5 \cdot \log_2(0.5) + 0.5 \cdot \log_2(0.5)) \\ &= -(0.5 \cdot (-1) + 0.5 \cdot (-1)) \\ &= -(-0.5 - 0.5) = 1 \text{ bit} \end{aligned}$$

需要 1 bit 来完整地写下

如果是不公平的硬币，那么最后的熵就小于 1 bit

- Claude Shannon showed: you cannot store the outcome of a random draw using fewer expected bits than the entropy without losing information.
- So units of entropy are **bits**; a fair coin flip has 1 bit of entropy.



- “坏”的叶子节点 (50% 是 A, 50% 是 B): 无论你抽到 A 还是 B, 都“有点意外” (因为你有一半的概率猜错)。所以它的“平均意外程度” (熵) 很高。
- “好”的叶子节点 (100% 是 A, 0% 是 B): 你抽到 A “毫不意外” (信息量为 0)。你永远抽不到 B。所以它的“平均意外程度” (熵) 为 0。

熵 = 信息的物理下限 (Entropy = The Physical Limit)
“熵”不仅是一个数学概念，它是一个物理上的硬限制。
• 熵 = “数据压缩的极限”。
• 它告诉你，要无损地存储来自某个概率分布的数据，你平均最少需要多少个 bit。

总熵下降得最多，就等于：
1. “不确定性消除得最多”
2. “获得的信息最多”

Entropy

- More generally, the entropy of a discrete random variable Y is given by

$$H(Y) = - \sum_{y \in Y} p(y) \log_2 p(y)$$

熵的通用公式

- “High Entropy”

- Variable has a uniform like distribution over many outcomes
- Flat histogram
- Values sampled from it are less predictable

- “Low Entropy”

- Distribution is concentrated on only a few outcomes
- Histogram is concentrated in a few areas
- Values sampled from it are more predictable

$H(Y)$: 代表变量 Y 的熵。这是最终想要的
的那个用来表示不确定性的数值

Y : 指一个随机变量

$\log_2(y)$: 指出这个结果的信息量或意外程度
以 bit 为单位。(通常一个概率价
的事件，会有很高的信息量 (熵很大))

$p(y)$: y 发生的概率

Σ : 对 Y 的所有可能结果都计算一遍

$p(y) \cdot \log_2 p(y)$

Entropy

- Suppose we observe partial information X about a random variable Y
 - For example, $X = \text{sign}(Y)$.
- We want to work towards a definition of the expected amount of information that will be conveyed about Y by observing X .
 - Or equivalently, the expected reduction in our uncertainty about Y after observing X .

高熵 (High Entropy)=高度混乱

变量的概率分布在许多结果上非常均匀

意思是：每一种可能性发生的概率都差不多大。

扁平的直方图(Flat histogram)

意思是：如果把所有可能性的概率画成柱状图，这些柱子都差不多高。

从中抽取的样本可预测性很低

意思是：你很难猜下一次会发生什么。

经典例子：掷一个公平的6面骰子。点数1到6的概率都是1/6。你完全无法预测下一次会掷出几，这就是高度不确定的（高熵）系统。

低熵 (Low Entropy)=高度有序

概率分布集中在少数几个结果上:

意思是：只有一两种可能性概率非常高，其他的可能性都微乎其微。

直方图集中在少数几个区域（有高峰）

意思是：柱状图上会有一两个高高耸起的柱子，其他的都非常矮。

从中抽取的样本可预测性很高

意思是：你很容易猜下一次会发生什么（大概率就是那个概率最高的）。

经典例子：掷一个作弊的骰子，它有90%的概率会掷出“6”。这个系统就非常有序，不确定性很低（低熵），因为你几乎总能猜对结果是“6”。

Entropy of a Joint Distribution

- Example: $X = \{\text{Raining, Not raining}\}, Y = \{\text{Cloudy, Not cloudy}\}$

$p(x,y)$ 联合概率, x,y 同时发生的概率.

$H(x,y)$ 所有 x,y 组合的总体不确定性

		Y	Cloudy	Not Cloudy
X	Raining		24/100	1/100
	Not Raining		25/100	50/100

$[\text{Raining, cloudy}] : -\frac{24}{100} \log_2 \frac{24}{100}$

$[\text{Raining, Not Cloudy}] : -\frac{1}{100} \log_2 \frac{1}{100}$

$[\text{Not Raining, Cloudy}] : -\frac{25}{100} \log_2 \frac{25}{100}$

$[\text{Not Raining, Not Cloudy}] : -\frac{50}{100} \log_2 \frac{50}{100}$

$$H(X,Y) = - \sum_{x \in X} \sum_{y \in Y} p(x,y) \log_2 p(x,y)$$
$$= - \frac{24}{100} \log_2 \frac{24}{100} - \frac{1}{100} \log_2 \frac{1}{100} - \frac{25}{100} \log_2 \frac{25}{100} - \frac{50}{100} \log_2 \frac{50}{100}$$
$$\approx 1.56 \text{ bits}$$

Specific Conditional Entropy 特定条件熵

我们不再看整个系统的混乱程度，而是看“当我们知道了某个信息之后，剩下的不确定性还有多少？”

- Example: $X = \{\text{Raining, Not raining}\}$, $Y = \{\text{Cloudy, Not cloudy}\}$

$H(Y|X=\text{raining})$ 是在 X (天气) = 下雨这个条件下，对 Y (是否多云) 的熵

因为这里只关注 $X=\text{raining}$ 的情况，所以完全忽略不下雨的情况

$X \backslash Y$	Cloudy	Not Cloudy
Raining	24/100	1/100
Not Raining	25/100	50/100

$$P(X=\text{raining}) = \frac{24}{100} + \frac{1}{100} = \frac{25}{100}$$

- What is the entropy of cloudiness Y , **given that it is raining**?

$$H(Y|X=x) = - \sum_{y \in Y} p(y|x) \log_2 p(y|x)$$

$$= - \frac{24}{25} \log_2 \frac{24}{25} - \frac{1}{25} \log_2 \frac{1}{25}$$

$$\approx 0.24 \text{ bits}$$

$$P(Y=\text{cloudy} | X=\text{Raining}) = \frac{P(\text{Raining, Cloudy})}{P(\text{Raining})}$$

$$= \frac{\frac{24}{100}}{\frac{25}{100}} = \frac{24}{25}$$

$$P(Y=\text{Not cloudy} | X=\text{Raining}) = \frac{P(\text{Raining, Not cloudy})}{P(\text{Raining})}$$

$$= \frac{\frac{1}{100}}{\frac{25}{100}} = \frac{1}{25}$$

- We used: $p(y|x) = \frac{p(x,y)}{p(x)}$ and $p(x) = \sum_y p(x,y)$ (sum in a row)

Conditional Entropy 条件熵

$H(Y|X)$ 就是计算 X 的所有可能性的特定熵，按它们发生的概率加权平均

	Cloudy	Not Cloudy
Raining	24/100	1/100
Not Raining	25/100	50/100

- The expected conditional entropy:

$$H(Y|X) = \sum_{x \in X} p(x) H(Y|X=x)$$

$$= - \sum_{x \in X} \sum_{y \in Y} p(x,y) \log_2 p(y|x)$$

$$H(Y|X) = P(\text{Raining}) \cdot H(Y|X=\text{Raining}) + P(\text{Not Raining}) \cdot H(Y|X=\text{Not Raining})$$

$$P(\text{raining}) = \frac{24}{100} + \frac{1}{100} = \frac{25}{100}$$

$$P(\text{not raining}) = \frac{25}{100} + \frac{50}{100} = \frac{75}{100}$$

Conditional Entropy

- Example: $X = \{\text{Raining, Not raining}\}$, $Y = \{\text{Cloudy, Not cloudy}\}$

熵 $H(Y|X=\text{Raining}) \approx 0.24 \text{ bit}$

$H(Y|X=\text{Not Raining})$

$$P(Y=\text{Cloudy} | X=\text{Not Raining}) = \frac{25}{100} = \frac{1}{4}$$

$$P(Y=\text{Not Cloudy} | X=\text{Not Raining}) = \frac{75}{100} = \frac{3}{4}$$

	Cloudy	Not Cloudy
Raining	24/100	1/100
Not Raining	25/100	50/100

$H(Y|X=\text{Not Raining})$

$$-\left[\left(\frac{1}{4}\right)\log_2\left(\frac{1}{4}\right) + \left(\frac{3}{4}\right)\log_2\left(\frac{3}{4}\right)\right] \approx 0.918 \text{ bits}$$

$$H(Y|X) = (0.25) \cdot (0.24) + (0.75) \cdot (0.918) \approx 0.7485 \text{ bits}$$

- What is the entropy of cloudiness Y , given the knowledge of whether or not it is raining?

$$\begin{aligned} H(Y|X) &= \sum_{x \in X} p(x) H(Y|X=x) \\ &= \frac{1}{4} H(\text{cloudiness} | \text{is raining}) + \frac{3}{4} H(\text{cloudiness} | \text{not raining}) \\ &\approx 0.75 \text{ bits} \end{aligned}$$

Conditional Entropy ☆

- Some useful properties

➤ H is always non-negative 熵永远非负的

➤ Chainrule: $H(X, Y) = H(X|Y) + H(Y) = H(Y|X) + H(X)$

➤ If X and Y independent, then X does not affect our uncertainty about Y : $H(Y|X) = H(Y)$ 如果 X 和 Y 是独立的 (一个的发生不会影响到另一个)

➤ But knowing Y makes our knowledge of Y certain: $H(Y|Y) = 0$

➤ By knowing X , we can only decrease uncertainty about Y : $H(Y|X) \leq H(Y)$

information gain 永远大于等于 0

决策树要做的, 找到那个能让 $H(Y|X)$ 最小 (即让信息增益最大) 的特征 X 来进行分裂

Information Gain

我对 Y (是否多云) 的原始不确定性, 减去我看到了 X (是否下雨) 之后, 对 Y 平均还剩下的不确定性。

	Cloudy	Not Cloudy
Raining	24/100	1/100
Not Raining	25/100	50/100

信息增益 (Information Gain): 这是它在决策树领域的常用名字。

互信息 (Mutual Information): 这是它在信息论里的名字。两者是完全等价的, 都衡量 X 和 Y 共享了多少信息。

- How much more certain am I about whether it's cloudy if I'm told whether it is raining? My uncertainty in Y minus my expected uncertainty that would remain in Y after seeing X. 如果我被告知是否下雨(X), 那么我对是否多云(Y)的不确定性会增加多少。
- This is called the **information gain** $IG(Y|X)$ in Y due to X, or the **mutual information** of Y and X. 这被称为由 X 带来的关于 Y 的信息增益, 或者 Y 和 X 的互信息。我们通常希望 $IG(Y|X)$ 越大越好, (对应 $H(Y|X)$ 最小)

$$IG(Y|X) = H(Y) - H(Y|X) \quad (1)$$

- If X is completely uninformative about Y: $IG(Y|X) = 0$ 如果 X 和 Y 相互独立, 毫无关系, $H(Y|X) = H(Y)$
- If X is completely informative about Y: $IG(Y|X) = H(Y)$ 如果 X 对 Y 提供了全部信息, 一旦你知道 X, Y 就 100% 确定了, $H(Y|X) = 0$

Revisiting Our Original Example

- Information gain measures the informativeness of a variable, which is exactly what we desire in a decision tree split!
- The information gain of a split: how much information (over the training set) about the class label Y is gained by knowing which side of a split you're on.

Revisiting Our Original Example

根熵 $H(Y)$ 分割前 整个数据集的熵 (纯度)

$$P(\text{oranges}) = \frac{5}{7} \quad P(\text{lemons}) = \frac{2}{7}$$

$$H(Y) = -\sum p \log_2 p = -\left(\frac{5}{7} \log_2 \frac{5}{7} + \frac{2}{7} \log_2 \frac{2}{7}\right) \approx 0.86$$

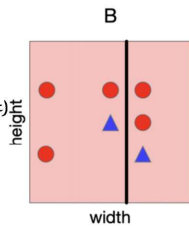
- What is the information gain of split B? Not terribly informative...

$H(Y|\text{left})$ 左侧

$$P(\text{oranges}|\text{left}) = \frac{3}{4}$$

$$P(\text{lemons}|\text{left}) = \frac{1}{4}$$

$$H(Y|\text{left}) = -\left(\frac{3}{4} \log_2 \frac{3}{4} + \frac{1}{4} \log_2 \frac{1}{4}\right) \approx 0.81$$



$H(Y|\text{right})$

$$P(\text{oranges}|\text{right}) = \frac{2}{3}$$

$$P(\text{lemons}|\text{right}) = \frac{1}{3}$$

$$H(Y|\text{right}) = -\left(\frac{2}{3} \log_2 \frac{2}{3} + \frac{1}{3} \log_2 \frac{1}{3}\right) \approx 0.92$$

$$H(Y|X) = P(\text{left}) \cdot H(Y|\text{left}) + P(\text{right}) \cdot H(Y|\text{right})$$

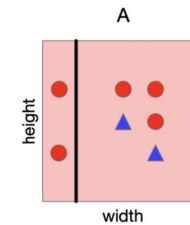
$$P(\text{left}) = \frac{4}{7} \quad P(\text{right}) = \frac{3}{7}$$

- Root entropy of class outcome: $H(Y) = -\frac{2}{7} \log_2 \left(\frac{2}{7}\right) - \frac{5}{7} \log_2 \left(\frac{5}{7}\right) \approx 0.86$
- Leaf conditional entropy of class outcome: $H(Y|\text{left}) \approx 0.81, H(Y|\text{right}) \approx 0.92$
- $IG(\text{split}) \approx 0.86 - \left(\frac{4}{7} \cdot 0.81 + \frac{3}{7} \cdot 0.92\right) \approx 0.006$

$$H(Y|X) = 0.86 - \left[\left(\frac{4}{7} \cdot 0.81\right) + \left(\frac{3}{7} \cdot 0.92\right)\right] = 0.86 - 0.857 \approx 0.003 \quad \text{很差的信息}$$

Revisiting Our Original Example

- What is the information gain of split A? Very informative!

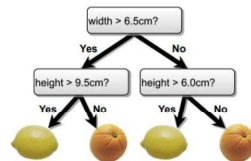
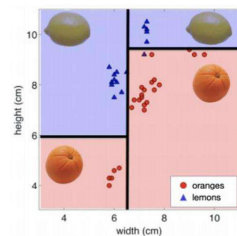


- Root entropy of class outcome: $H(Y) = -\frac{2}{7} \log_2 \left(\frac{2}{7}\right) - \frac{5}{7} \log_2 \left(\frac{5}{7}\right) \approx 0.86$
- Leaf conditional entropy of class outcome: $H(Y|\text{left}) \approx 0, H(Y|\text{right}) \approx 0.97$
- $IG(\text{split}) \approx 0.86 - \left(\frac{2}{7} \cdot 0 + \frac{5}{7} \cdot 0.97\right) \approx 0.17!$

Constructing Decision Trees

信息增益 = 熵的下降量

最好的切分, 就是让系统的总熵熵度下降最多的切分。



- At each level, one must choose: 在每一层都必须要考虑的事情:
 - Which feature to split. 1. 基于什么特征来分割
 - Possibly where to split it. 2. 分割的划分线是什么。
- Choose them based on how much information we would gain from the decision! (choose feature that gives the highest gain) 选择提供最高信息增益的特征

Decision Tree Construction Algorithm

决策树构建公式。

Greedy: 每次都只选当前带来最大 information gain 的切法。

Recursive: 重复一模一样的切分流程 (不断细分) ↑

- Simple, greedy, recursive approach, builds up tree node-by-node

- Pick a feature to split at a non-terminal node 计算每一种切法带来的IG. (Greedy)
- Split examples into groups based on feature value 采用增益最高的切法
- For each group: 继续 / 终止条件 ↓
 - If no examples – return majority from parent 如果一个节点的例子=0, 则不用再细分, 并标上对应
 - Else if all examples in same class – return class 如果这一组仍然混乱, 回到步骤1, 检查, 重新找能带来最大IG的方法。
 - Else loop to step 1

- Terminates when all leaves contain only examples in the same class or are empty.

BacktoOurExample

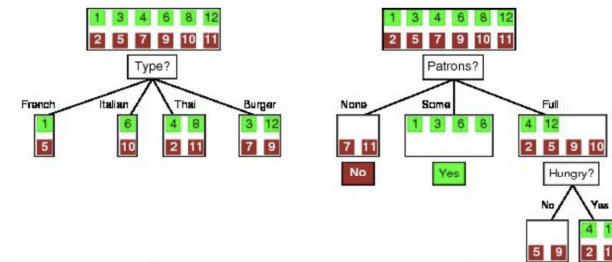
Example	Input Attributes										Goal
	Alt	Bar	Fri	Hun	Pat	Price	Rain	Res	Type	Est	WillWait
x ₁	Yes	No	No	Yes	Some	\$\$\$	No	Yes	French	0-10	y ₁ = Yes
x ₂	Yes	No	No	Yes	Full	\$	No	No	Thai	30-60	y ₂ = No
x ₃	No	Yes	No	No	Some	\$	No	No	Burger	0-10	y ₃ = Yes
x ₄	Yes	No	Yes	Yes	Full	\$	Yes	No	Thai	10-30	y ₄ = Yes
x ₅	Yes	No	Yes	No	Full	\$\$\$	No	Yes	French	>60	y ₅ = No
x ₆	No	Yes	No	Yes	Some	\$\$	Yes	Yes	Italian	0-10	y ₆ = Yes
x ₇	No	Yes	No	No	None	\$	Yes	No	Burger	0-10	y ₇ = No
x ₈	No	No	No	Yes	Some	\$\$	Yes	Yes	Thai	0-10	y ₈ = Yes
x ₉	No	Yes	Yes	No	Full	\$	Yes	No	Burger	>60	y ₉ = No
x ₁₀	Yes	Yes	Yes	Yes	Full	\$\$\$	No	Yes	Italian	10-30	y ₁₀ = No
x ₁₁	No	No	No	No	None	\$	No	No	Thai	0-10	y ₁₁ = No
x ₁₂	Yes	Yes	Yes	Yes	Full	\$	No	No	Burger	30-60	y ₁₂ = Yes

1. Alternate: whether there is a suitable alternative restaurant nearby.
2. Bar: whether the restaurant has a comfortable bar area to wait in.
3. Fri/Sat: true on Fridays and Saturdays.
4. Hungry: whether we are hungry.
5. Patrons: how many people are in the restaurant (values are None, Some, and Full).
6. Price: the restaurant's price range (\$, \$\$, \$\$\$).
7. Raining: whether it is raining outside.
8. Reservation: whether we made a reservation.
9. Type: the kind of restaurant (French, Italian, Thai or Burger).
10. WaitEstimate: the wait estimated by the host (0-10 minutes, 10-30, 30-60, >60).

Features:

[from: Russell & Norvig]

FeatureSelection



$$IG(Y) = H(Y) - H(Y|X)$$

$$IG(\text{type}) = 1 - \left[\frac{2}{12} H(Y|\text{Fr.}) + \frac{2}{12} H(Y|\text{It.}) + \frac{4}{12} H(Y|\text{Thai}) + \frac{4}{12} H(Y|\text{Bur.}) \right] = 0$$

$$IG(\text{Patrons}) = 1 - \left[\frac{2}{12} H(0,1) + \frac{4}{12} H(1,0) + \frac{6}{12} H\left(\frac{2}{6}, \frac{4}{6}\right) \right] \approx 0.541$$

What Makes a Good Tree?

- Not too small: need to handle important but possibly subtle distinctions in data. *过多的问题太少, 这样的树太“天真”, 忽略了数据中那些重要或微妙的区别. (欠拟合).*
- Not too big: *不能为了区分进行区分而专门增加很多没有泛用性的分割*
 - Computational efficiency (avoid redundant, spurious attributes) *避免过度拟合.*
 - Avoid over-fitting training examples *可塑性 树形的划分结构要清晰*
 - Human interpretability *树越大, 计算效率越低.*
- “Occam’s Razor”: find the simplest hypothesis that fits the observations. *增加新的分支时, 要考虑增加的计算成本和所得成果的关系.*
 - Useful principle, but hard to formalize (how to define simplicity?)
 - See Domingos, 1999, “The role of Occam’s razor in knowledge discovery”
- We desire small trees with informative nodes near the root

如果有两棵树, 一棵 5 层的树能达到 95% 的准确率, 一棵 50 层的树能达到 96% 的准确率。我们应该毫不犹豫地选择那棵 5 层的!

Decision Tree Miscellany

- Problems: *① 树越往下走, 数据越少, 最后可能只通过 2 个样本来进行分割, 这种方法不够泛用性. ② 过拟合 ③ 贪心算法不一定能得到全局最优解*
 - You have exponentially less data at lower levels
 - Too big of a tree can overfit the data
 - Greedy algorithms don’t necessarily yield the global optimum
- Handling continuous attributes
 - Split based on a threshold, chosen to maximize information gain

Comparison to some other classifiers

- ① 可解释性强: 一棵决策树就是 if-then-else
 ② Fast at test time: 预测时速度很快 (只需跟着走而不用走一遍)
 ③ 能简单处理脏数据
 • 离散特征: 天气=晴/阴/雨 不用 'one-hot' 编码
 • 缺失值: 可以处理我不知道这个问题答案"的情况
 • 未缩放数据: 决策树不在乎数据是具体哪个单位, 只关注分割线 (神经网络对尺度很敏感)

Advantages of decision trees over KNNs and neural nets

- Simple to deal with discrete features, missing values, and poorly scaled data
- Fast at test time
- More interpretable

Advantages of KNNs over decision trees

- Few hyperparameters 更少的超参数
- Can incorporate interesting distance measures (e.g. shape contexts)

Advantages of neural nets over decision trees

神经网络能处理复杂方式交互的特征。

- Able to handle attributes/features that interact in very complex ways (e.g. pixels)
 决策树的切分只能“横竖切分”，很难学习到对角线这样的组合特征。
 神经网络可以做到像素级学习。

Comparison to some other classifiers

- We've seen many classification algorithms. 将多种分类器进行组合, 变成“集成方法”
- We can combine multiple classifiers into an **ensemble**, which is a set of predictors whose individual decisions are combined in some way to classify new examples
 ➤ E.g., (possibly weighted) majority vote (加权) 少数服从多数
- For this to be nontrivial, the classifiers must differ somehow, e.g.
 - Different algorithm 包含多种不同的分类算法 (用相同的算法就没意义了)
 - Different choice of hyperparameters 用不同的超参数设定
 - Trained on different data 不同训练集上学习的分类器
 - Trained with different weighting of the training examples 用不同的权重训练
- Next lecture, we will study some specific ensembling techniques.

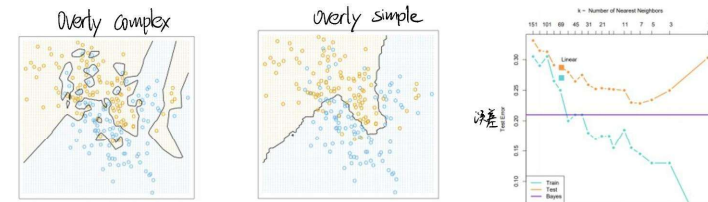
- Today, we deepen our understanding of generalization through a bias-variance decomposition. 偏差-方差分解: 诊断一个模型为什么会犯错的黄金标准。

➤ This will help us understand ensembling methods.

高偏差 (High bias): 模型太天真, 太简单了, 欠拟合
低偏差 (low bias): 模型太敏感, 过拟合

Bias-Variance Decomposition

- Recall that overly simple models underfit the data, and overly complex models overfit.



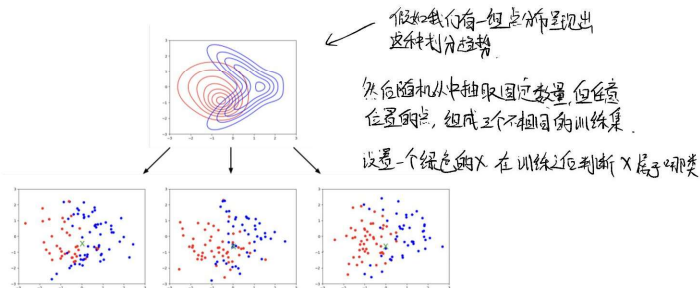
- We can quantify this effect in terms of the **bias/variance decomposition**.

➤ Bias and variance of what?

由左到右, 模型越来越复杂
蓝线 (训练误差): 越往右, 模型越复杂, 过拟合导致训练误差下降
橙线 (测试误差): 左侧模型简单, 训练误差高
右侧模型复杂, 训练误差低, 但泛化能力差, 测试误差重新升高

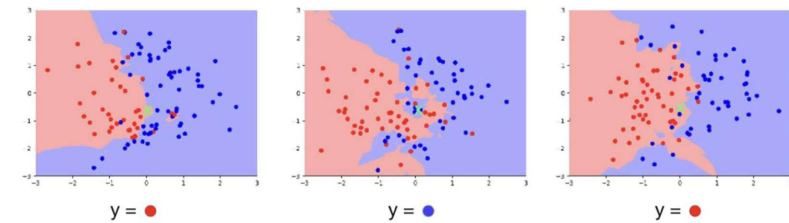
Bias-Variance Decomposition: Basic Setup

- Suppose the training set $\mathcal{D}$ consists of pairs (x_i, t_i) sampled **independent and identically distributed (i.i.d)** from a single **data generating distribution** p_{sample} .
- Pick a fixed query point $\mathbf{x}$ (denoted with a green $\mathbf{x}$).
- Consider an experiment where we sample lots of training sets independently from p_{sample} .



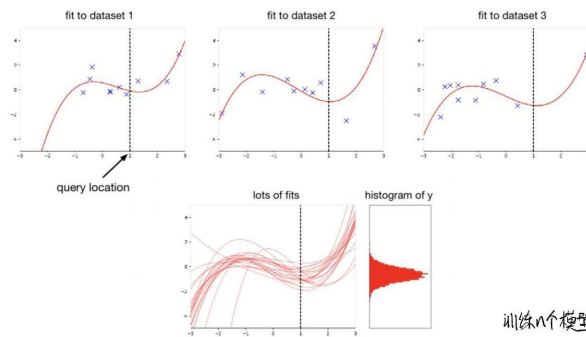
Bias-Variance Decomposition: Basic Setup

- Let's run our learning algorithm on each training set, and compute its prediction y at the query point $\mathbf{x}$.
- we can view y as a random variable, where the randomness comes from the choice of training set.
- The classification accuracy is determined by the distribution of y .



Bias-Variance Decomposition: Basic Setup

Here is the analogous setup for regression: 回归问题中, 当 $x=1$ 时, y 的期望值.



训练 n 个模型, 会得到 n 个不同的 y 值.

Since y is a random variable, we can talk about its expectation, variance, etc.

Bias-Variance Decomposition: Basic Setup

- Recap of basic setup: 可以把预测值 y 视为一个随机变量, y 的值完全取决于对应的训练集.
 - Fix a query point x .
 - Repeat:
 - Sample a random training dataset $\mathcal{D}$ i.i.d. from the data generating distribution p_{sample} .
 - Run the learning algorithm on $\mathcal{D}$ to get a prediction y at x .
 - Sample the (true) target from the conditional distribution $p(t|x)$.
 - Compute the loss $L(y, t)$. 比较预测结果 y 和真实结果 t .
- Notice: y is independent of t . y 只取决于数据集 $\mathcal{D}$. 在寻找目标 x 的结果时的平均损失“期望损失”
- This gives a distribution over the loss at x , with expectation $\mathbb{E}[L(y, t)|x]$.
- For each query point x , the expected loss is different. We are interested in minimizing the expectation of this with respect to $x \sim p_{\text{sample}}$. 最小化这个平均损失在其他所有 x 上的总平均值.

BayesOptimality 贝叶斯最优

($y-t$)² 越小, 代表预测越好.

- For now, focus on squared error loss, $L(y, t) = \frac{1}{2}(y - t)^2$. 损失函数 (平方误差损失)
- A first step: suppose we knew the conditional distribution $p(t|x)$. What value y should we predict? $p(t|x)$: 所有可能的标准答案 t 的概率分布 (50% A, 50% B). 面对不确定的 t , 必须选个
 ➤ Here, we are treating t as a random variable and choosing y . 固定的 y 值作为预测.
- Claim:** $y_* = \mathbb{E}[t|x]$ is the best possible prediction. 最好的预测值 y_* 是 t 在给定 x 下的期望值 ($\mathbb{E}$). 也就是平均值. 例如 (房价: 30% 100万, 60% 101万, 10% 102万) $\Rightarrow (0.3 \times 100) + (0.6 \times 101) + (0.1 \times 102) = 100.8$ 万.
- Proof:**

$$\begin{aligned}
 \mathbb{E}[(y-t)^2|x] &= \mathbb{E}[y^2 - 2yt + t^2|x] \\
 &= y^2 - 2y\mathbb{E}[t|x] + \mathbb{E}[t^2|x] \\
 \text{Var}[t|x] &= \mathbb{E}[t^2|x] - (\mathbb{E}[t|x])^2 \\
 &= \mathbb{E}[t^2|x] - y_*^2 \\
 \mathbb{E}[t^2|x] &= \text{Var}[t|x] + y_*^2 \\
 \mathbb{E}[(y-t)^2|x] &= y^2 - 2yy_* + y_*^2 + \text{Var}[t|x] \\
 &= (y - y_*)^2 + \text{Var}[t|x]
 \end{aligned}$$

y_* 应该瞄准的靶心, (调整 y)
 $\text{Var}[t|x]$ 要损失掉的噪声. (无法避免)
 让 $y - y_*$ 变为 0 是 optimal result. $\text{Var}[t|x]$ (噪声) 是个常数

BayesOptimality

$$\mathbb{E}[(y-t)^2|x] = (y - y_*)^2 + \text{Var}[t|x]$$

- 在上面, 我们假设知道 $p(t|x)$, 也就是 y_* 的精确值, 从而让 $y = y_*$.
- The first term is nonnegative, and can be made 0 by setting $y = y_*$.
- The second term corresponds to the inherent unpredictability, or **noise**, of the targets, and is called the **Bayes error**. 噪声 y 是一个随机变量, 从一个随机训练集中学来的.
 ➤ This is the best we can ever hope to do with any learning algorithm. An algorithm that achieves it is **Bayes optimal**.
 ➤ Notice that this term doesn't depend on y .
- This process of choosing a single value y_* based on $p(t|x)$ is an example of **decision theory**.

BayesOptimality

- Now return to treating y as a random variable (where the randomness comes from the choice of dataset).
- We can decompose out the expected loss (suppressing the conditioning on $\mathbf{x}$ for clarity):

$$\begin{aligned}
 \mathbb{E}[(y - t)^2] &= \mathbb{E}[(y - y_*)^2] + \text{Var}(t) \\
 &= \mathbb{E}[y_*^2 - 2y_*y + y^2] + \text{Var}(t) \\
 &= y_*^2 - 2y_*\mathbb{E}[y] + \mathbb{E}[y^2] + \text{Var}(t) \\
 &= y_*^2 - 2y_*\mathbb{E}[y] + \mathbb{E}[y^2] + \text{Var}(y) + \text{Var}(t) \\
 &= \underbrace{(y_* - \mathbb{E}[y])^2}_{\text{bias}} + \underbrace{\text{Var}(y)}_{\text{variance}} + \underbrace{\text{Var}(t)}_{\text{Bayes error}}
 \end{aligned}$$

偏差
方差
贝叶斯误差

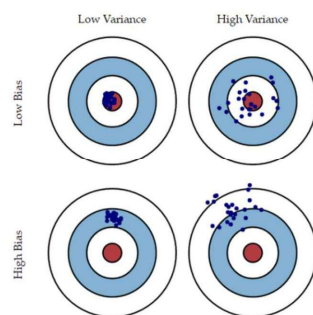
BayesOptimality

bias: 期望预测 $\mathbb{E}[y]$ 距离真实值 y_* 有多远。在所有模型上, 在平均意义上给出的
 最优预测, 如果该法本身有偏差
 variance: $\text{Var}(y)$ 在所有模型预测值 y 在它们 自己的平均值 $\mathbb{E}[y]$ 周围摇摆程度 (偏差: 过拟合
 那么最终结果 $\mathbb{E}[y]$ 也会偏离 y_* 何题: 拟合
 Bayes error: $\text{Var}(t)$ 目标值固有的不可预测性 (噪声)

- We just split the expected loss into three terms:
 - **bias**: how wrong the expected prediction is (corresponds to underfitting)
 - **variance**: the amount of variability in the predictions (corresponds to overfitting)
 - Bayes error: the inherent unpredictability of the targets
- Even though this analysis only applies to squared error, we often loosely use “bias” and “variance” as synonyms for “underfitting” and “overfitting”.

Bias and Variance

- Throwing darts = predictions for each draw of a dataset



- Be careful, what doesn't this capture?
 - We average over points x from the data distribution.